Advanced Data Management
Quiz Refresher: Chapters 1–3

DBMS internals, storage and file organisation, indexing. Rapid recall, multiple choice with justification, and six midterm-style problems, all answered at the end

Advanced Data Management

2026-10-27

How to use this document

Three rounds, answers in Part D. Round A is 30 one-line recall questions: cover the answers, go fast, and mark every one you hesitate on. Round B is eight multiple-choice questions in the mock's format, where the justification is worth as much as the letter. Round C is six problems in the style of the mock's Parts 2 and 3, with numbers to compute. Budget about 25 minutes for A and B together and 60 minutes for C. Every number in the answers was computed by hand and checked against the formulas in the chapters.

Round	Content	Time
A	30 rapid-recall questions (10 per chapter)	15 min
B	8 multiple choice, letter + one-line justification	10 min
C	6 midterm-style problems	60 min
D	Answers	

Round A: rapid recall

Chapter 1: DBMS internals and the life of a query

A1. Name the five subsystems of a DBMS.
A2. Name the five stages a query passes through, in order.
A3. Which stage detects a misspelled column name, and what does it consult to do so?
A4. What does "declarative" mean for SQL, and what is the consequence for the DBMS?
A5. What three methods does every operator expose in the iterator model?
A6. What is pipelining, and which operator in a typical plan breaks it?
A7. In an `EXPLAIN` tree, which line runs first?

A8. Under a `Hash Join`, which child is the build side?
A9. What is predicate pushdown, and why does the optimiser do it?
A10. OLTP versus OLAP: one sentence each, and which storage model suits each.

Chapter 2: file organisation and storage

A11. What is the unit of cost in Chapters 2–5, and what is the page size in PostgreSQL?
A12. Why is counting pages a fair approximation of time?
A13. Name two blind spots of the page-count model.
A14. Heap file of B pages: cost of a full scan, of finding a present unique key on average, of finding an absent key.
A15. Give the formula for records per page (header 24 bytes, slot 4 bytes).
A16. Why does a slotted page store offsets in the slot directory rather than the tuples in fixed positions?
A17. Equality search cost on a heap, a sorted file, a hashed file.
A18. Why is a hashed file useless for range queries?
A19. Why do sequential reads beat random reads on a hard disk, and what does the DBMS do about it?
A20. Give two things the DBMS knows that the OS page cache does not.

Chapter 3: indexing

A21. What is an index, in one sentence?
A22. Define selectivity.
A23. Why does having an index not guarantee the optimiser uses it?
A24. Height of a B+ tree with fanout f over N entries; cost of a point lookup.
A25. Why are B+ tree leaves linked?
A26. What happens when you insert into a full leaf?
A27. Clustered versus secondary index: the difference and the consequence for range scans.
A28. State the leftmost-prefix rule.
A29. When would you use a partial index? An expression index? A covering index?
A30. Give the three costs of having an index.

Round B: multiple choice, letter plus justification

Each question is 2 points: 1 for the letter, 1 for a one-line justification (a number, a formula or a sentence).

B1. **Absent key.** A heap file has $B = 4{,}000$ pages and no index. Cost to establish that `id = 99` is *not* in the table?
(a) 1 (b) $\log_2 4000 \approx 12$ (c) 2,000 (d) 4,000

B2. **Records per page.** 8 KB pages, 24-byte page header, 150-byte fixed records, 4-byte slots. Records per page?
(a) 51 (b) 53 (c) 54 (d) 55

B3. **B+ tree lookup.** A B+ tree with fanout 200 indexes 5,000,000 rows. Page reads for one point lookup, including the heap row?
(a) 3 (b) 4 (c) 5 (d) 23

B4. **Leftmost prefix.** There is one index, on `(country, city)`. Which predicate cannot use it?
(a) `country = 'MA'` (b) `country = 'MA' AND city = 'Fes'` (c) `city = 'Fes'` (d) `country > 'M'`

B5. **Which stage.** `SELECT * FROM ordrs;` fails with "relation ordrs does not exist". Which stage reports it?
(a) parsing (b) binding (c) optimisation (d) execution

B6. **Column store.** A table has 30 columns of similar width and one million rows. `SELECT avg(price)` on a column store reads roughly what fraction of the pages a row store reads, before compression?
(a) the same (b) about 1/30 (c) about 1/2 (d) about 30 times more

B7. **Hash index.** A hash index helps which of these?
(a) `ts BETWEEN a AND b` (b) `ORDER BY email` (c) `email = 'x@y.com'` (d) `email LIKE 'a%'`

B8. **Sorted file insert.** Inserting one row into a sorted file of B pages costs, in the worst case:
(a) about 1 (b) $\log_2 B$ (c) up to B (d) 0, inserts are free

Round C: midterm-style problems

Problem: C1. Storage estimation [8 pts]

A table `tickets(id bigint, event_id int, seat varchar, price numeric, sold_at timestamp)` holds 3,000,000 rows. Column sizes: 8, 4, 12 (average, including its length byte), 8, 8 bytes. PostgreSQL adds a 24-byte tuple header and aligns each tuple to 8 bytes; each tuple costs a 4-byte slot; the page header is 24 bytes of an 8 KB page.

(a) Compute the tuple size, the records per page, the number of pages and the table size in MB.
(b) Give the cost in page reads of: a full scan; finding `id = 1234567` with no index (average); finding `id = -1` with no index.
(c) At 500 MB/s sequential throughput, how long does a cold full scan take?
(d) A colleague adds a fixed 100-byte `notes` column. Recompute the pages and the scan time, and state the factor by which the scan got slower.
(e) Which PostgreSQL calls check the page count empirically?

Problem: C2. Choosing a file organisation [6 pts]

For each workload, choose heap, sorted or hashed, and justify with the cost table (B pages).

(a) A product catalogue looked up only by exact `sku`; updated once a night.

(b) An append-only sensor log, read only by full scans for monthly reports.

(c) A ledger queried almost exclusively as "all entries between two dates".

(d) The log of (b), but now also queried by exact `sensor_id` thousands of times per hour. What does a real system do instead of changing the file organisation?

Problem: C3. B+ tree arithmetic [8 pts]

The `tickets` table of C1 (3,000,000 rows, 25,000 pages) gets a B+ tree on `id`. Fanout is 150 and a leaf also holds 150 entries.

(a) Height of the tree, with the calculation. Page reads for one lookup by `id`, including the heap row.

(b) A range on `id` matches 2,000 rows. Estimate the index-scan cost (descend, leaves, heap fetches) and compare with the sequential scan.

(c) Another range matches 300,000 rows (10% of the table). Same comparison. Which plan does the optimiser pick?

(d) Estimate the crossover: below what fraction of the table does the index scan win? Explain why the true crossover is even lower than your estimate.

(e) Toy insert: leaves hold 4 keys; a leaf holds (10, 20, 30, 40) and its parent is (10 | 50). Insert 25. Show the leaves and the parent afterwards.

Problem: C4. Index design [8 pts]

Same table, now with `status` (`'sold'` 95%, `'reserved'` 4%, `'refunded'` 1%) and `buyer_email`. For each pattern give the `CREATE INDEX`, the index type, and a one-line justification. Identify the pattern no index will help, and prove it with a selectivity number.

(a) `WHERE buyer_email = $1`

(b) `WHERE sold_at BETWEEN $1 AND $2`

(c) `WHERE status = 'refunded'`

(d) `WHERE status = 'sold'`

(e) `WHERE lower(buyer_email) = $1`

(f) `WHERE event_id = $1 AND sold_at > $2`

(g) `ORDER BY sold_at DESC LIMIT 10`

Problem: C5. Reading a plan [8 pts]

```
HashAggregate  (rows=5)
  Group Key: e.city
  ->  Hash Join  (rows=120)  (actual rows=410000)
```

```
      Hash Cond: (t.event_id = e.id)
      ->  Seq Scan on tickets t  (rows=3000000)
      ->  Hash
            ->  Seq Scan on events e  (rows=4)  (actual rows=9000)
                  Filter: (country = 'MA')
```

(a) Which node runs first, and which side of the join is built in memory?

(b) Why is the `country` filter below the join rather than above it? Name the technique and the stage that applied it.

(c) Identify the node whose estimate is most wrong, give the factor, and state the most likely cause and the command that fixes it.

(d) Before this plan existed, the query text had a typo: `countr = 'MA'`. Which stage would have rejected it, and why not the parser?

(e) Suppose a second session updates `events` while this query runs and one of the two changes is lost. Which subsystem is responsible?

Problem: C6. Discussion [7 pts]

(a) The OS already caches file pages. Give three reasons a DBMS manages its own buffer pool anyway, and for each say what breaks or slows down with the OS alone. [4 pts]

(b) "A B+ tree lookup is $O(\log_2 N)$, so for a million rows it costs about 20 page reads." Explain what is wrong with this sentence and give the right number. [3 pts]

Round D: answers

Round A

A1. Connection/session manager, query processor, storage manager, transaction manager, catalog.

A2. Parse, bind, rewrite (logical plan), optimise (physical plan), execute.

A3. Binding, by looking the name up in the catalog. The parser checks grammar only.

A4. You state *what* you want, not *how*; the DBMS must choose the plan itself, which is what the optimiser does.

A5. `open`, `next`, `close`.

A6. Tuples flow one at a time up the tree without materialising intermediate results. A sort (and the build side of a hash join) must consume all input before producing any, so it breaks the pipeline.

A7. The most deeply indented line; read the tree bottom-up.

A8. The child under the `Hash` node; the other child is probed.

A9. Moving a filter below a join so it runs on the smaller input first; fewer rows reach every operator above.

A10. OLTP: many small reads and writes of whole rows, row storage. OLAP: few large scans of a few columns over many rows, column storage.

A11. The number of page reads and writes; 8 KB.

A12. A disk page read costs millions of CPU cycles, so pages are where the time goes, and they are easy to count by hand.

A13. It treats random and sequential reads as equal; it ignores caching (Chapter 6); it ignores CPU-heavy work such as in-memory sorting.

A14. B; about $B/2$; B always, because you must read every page to be sure.

A15. $\lfloor (8192 - 24)/(\text{record} + 4) \rfloor = \lfloor 8168/(\text{record} + 4) \rfloor$.

A16. So tuples can move inside the page during compaction without changing their (page, slot) id, which indexes rely on, and to allow variable-length tuples.

A17. Heap B (average $B/2$); sorted $\log_2 B$; hashed about 1.

A18. Adjacent keys hash to unrelated buckets, so a range touches every bucket: cost B.

A19. A random read moves the disk arm (about 10 ms); sequential reads stream. The DBMS lays related rows out contiguously, reads in large runs and prefetches.

A20. The access pattern (a one-pass scan should not evict hot pages) and the required write ordering (log before data, for durability).

A21. A separate sorted (or hashed) structure beside the heap that maps key values to row ids.

A22. The fraction of the table's rows a predicate returns.

A23. An index scan costs about one random page read per matching row; past a few percent of the table, B sequential reads are cheaper, and the optimiser picks the scan.

A24. $h = \lceil \log_f N \rceil$, typically 3–4; a point lookup costs $h + 1$ reads (one more

for the heap page).

A25. So a range query descends once and walks the chain of leaves in key order instead of re-searching per key.

A26. The leaf splits into two half-full leaves and the middle key is pushed up to the parent; if the parent is full it splits too, and the tree grows only when the root splits.

A27. Clustered: heap order follows the key, so a range reads adjacent heap pages. Secondary: heap order is unrelated, so each match is a random page read.

A28. A multi-column index on (a, b, c) serves predicates on a leading prefix: a; a, b; a, b, c. It cannot serve b alone, and after a range on a the later columns are only filters.

A29. Partial: the query always has the same selective predicate (`WHERE status = 'active'`). Expression: the query filters on `lower(name)`. Covering: the query needs only indexed columns, so the heap is never touched (index-only scan).

A30. Disk space; write amplification (every insert, update of the key, and delete also touches the index); optimiser effort and more chances for a wrong plan.

Round B

B1. **(d) 4,000.** An absent key forces a read of every page; $B/2$ is the average only for a present unique key.

B2. **(b) 53.** $\lfloor 8168/(150+4) \rfloor = \lfloor 53.04 \rfloor = 53$. Forgetting the slot gives 54; forgetting the floor is the other trap.

B3. **(b) 4.** $200^3 = 8{,}000{,}000 \geq 5{,}000{,}000$, so height 3; plus one heap read $= 4$. (d) is the $\log_2$ trap: $\log_2 5 \times 10^6 \approx 23$ is for binary trees, not fanout 200.

B4. **(c).** `city` is not a leading column; the index is sorted by `country` first, so `city` values are scattered. (d) is a range on the leading column, which the index serves.

B5. **(b) binding.** `ordrs` is a syntactically valid identifier; only the catalog lookup reveals there is no such table.

B6. **(b) about 1/30.** The column store reads only the `price` column's pages; the row store must read every page because every row's `price` sits beside 29 other columns.

B7. **(c).** Hashing destroys order, so ranges, sorts and prefix matches cannot use it; only equality on the whole key.

B8. **(c) up to B.** The row must go in key order, so on average half the file shifts; in the worst case all of it.

Round C

Solution: C1. Storage estimation

(a) Columns $8 + 4 + 12 + 8 + 8 = 40$ bytes; plus the 24-byte header $= 64$, already a multiple of 8. With the 4-byte slot each record costs 68 bytes. Records per page $= \lfloor 8168/68 \rfloor = \lfloor 120.1 \rfloor = 120$. Pages $= \lceil 3{,}000{,}000/120 \rceil = 25{,}000$. Size $= 25{,}000 \times 8$ KB $= 200{,}000$ KB ≈ 200 MB.
(b) Full scan 25,000. Present unique key, average $B/2 = 12{,}500$. Absent key 25,000: every page must be read to prove absence.
(c) 200 MB/500 MB/s $= 0.4$ s.
(d) Record $40 + 100 = 140$, plus header 164, aligned to 8 $\rightarrow$ 168, plus slot 172. Records per page $\lfloor 8168/172 \rfloor = \lfloor 47.5 \rfloor = 47$. Pages $\lceil 3{,}000{,}000/47 \rceil = 63{,}830$, about 511 MB, scan about 1.0 s. The scan got $63{,}830/25{,}000 \approx 2.55\times$ slower. Adding a column is never free: every query that scans pays for it even if it never reads `notes`.
(e) `SELECT pg_relation_size('tickets') / 8192;` for the page count, and `SELECT relpages, reltuples FROM pg_class WHERE relname = 'tickets';` for the planner's view of it (after `ANALYZE`). To see rows per page directly: `SELECT count(*) FROM heap_page_items(get_raw_page('tickets', 0));` from `pageinspect`. A large gap from the estimate means dead tuples (`VACUUM`) or TOAST.

Solution: C2. Choosing a file organisation

(a) Hashed. Equality search costs about 1 page versus $\log_2 B$ sorted and $B/2$ heap; the nightly update is a batch, so the hashed file's weakness (reorganising on growth) is irrelevant.
(b) Heap. Inserts cost about 1 page; a sorted file would cost up to B per insert. Reads are full scans, which cost B in every organisation, so nothing is lost.
(c) Sorted by date. Range search costs $\log_2 B+$ matching pages, versus B for heap and B for hashed, which scatters adjacent dates across buckets.
(d) It keeps the heap and adds an index on `sensor_id`: a B+ tree gives $h + 1 \approx 4$ reads per lookup while inserts stay cheap (heap page plus one leaf page). Changing the file organisation would trade the write cost for the read cost; an index pays a small write cost for both.

Solution: C3. B+ tree arithmetic

(a) $150^2 = 22{,}500$ and $150^3 = 3{,}375{,}000 \geq 3{,}000{,}000$, so height $= \lceil \log_{150} 3 \times 10^6 \rceil = 3$. A point lookup reads 3 index pages plus 1 heap page $= 4$, against 12,500 on average for a heap scan: about 3,000$\times$ cheaper.
(b) Descend 3; leaves $\lceil 2{,}000/150 \rceil = 14$; heap fetches up to 2,000 (one random page per row, since `id` order is unrelated to heap order). Total about $3 + 14 + 2{,}000 \approx 2{,}017$ reads versus 25,000 sequential: the index wins by about 12$\times$.

(c) Descend 3; leaves 300,000/150 = 2,000; heap fetches up to 300,000. About 302,000 random reads versus 25,000 sequential reads: the scan wins by about 12× even though it reads 90% "useless" rows. The optimiser picks the sequential scan.
(d) The index scan costs roughly one read per match, the scan costs $B =$ 25,000. They tie near 25,000 matches, which is 25,000/3,000,000 $\approx 0.8\%$ of the table. The true crossover is lower because the index reads are random (each one a seek on a disk, and a cache miss in any case) while the scan's are sequential, so the planner charges each random page several times more than a sequential one. Lab 3, Part 4 finds it experimentally around one or two percent at best.
(e) The leaf (10, 20, 30, 40) is full. Split: (10, 20) and (25, 30, 40); push the first key of the new right leaf, 25, into the parent. Parent becomes (10 | 25 | 50). In a B+ tree 25 stays in the leaf as well, since every key must appear at leaf level.

Solution: C4. Index design

	Index	Type	Why
(a)	`CREATE UNIQUE INDEX ON tickets(buyer_email)`	B-tree	Exact match returning one row: $h + 1 \approx 4$ reads instead of 25,000.
(b)	`CREATE INDEX ON tickets(sold_at)`	B-tree	Ranges need order: descend once, walk the leaves. A hash index cannot do this.
(c)	`CREATE INDEX ON tickets(status) WHERE status = 'refunded'`	partial B-tree	1% of rows, 30,000: index scan about 30,000 reads versus 25,000 scan is borderline, but the partial index is tiny (30,000 entries) and a bitmap heap scan sorts the fetches into page order, so it wins.
(d)	none		See below.
(e)	`CREATE INDEX ON tickets(lower(buyer_email))`	expression B-tree	The index on `buyer_email` sorts by the raw value; `lower()` is a different key.
(f)	`CREATE INDEX ON tickets(event_id, sold_at)`	multi-column B-tree	Equality on the leading column, then a range on the second: within one `event_id` run the `sold_at` values are contiguous. The reverse order would not serve the equality.
(g)	the index of (b)	B-tree	Walk the leaf chain backwards from the right end, stop after 10: about $h + 1$ pages, no sort.

The pattern no index helps: (d). `status = 'sold'` has selectivity 0.95, so $0.95 \times 3{,}000{,}000 = 2{,}850{,}000$ rows. An index scan would cost up to 2.85 million random page reads on a heap of only 25,000 pages: each page re-read about 114 times. The sequential scan reads 25,000 pages once. No index can beat that; the planner will scan.

Solution: C5. Reading a plan

(a) The deepest node: `Seq Scan on events` with its filter. Its output is hashed in memory (the build side); then `Seq Scan on tickets` is probed against the hash table.
(b) Predicate pushdown, done in the rewrite stage (logical plan) and confirmed by the optimiser. The filter mentions only `events` columns, so it can run before the join; it shrinks the build side from all events to the Moroccan ones, and every operator above sees fewer rows.
(c) `Seq Scan on events`: estimated 4, actual 9,000, off by 2,250×. The error propagates: the join estimate 120 versus actual 410,000. Cause: stale statistics (the table grew or the `country` distribution changed since the last `ANALYZE`). Fix: `ANALYZE events;` then re-run `EXPLAIN ANALYZE`.
(d) Binding. `countr` is grammatically a valid identifier, so parsing succeeds; the binder looks it up in the catalog, finds no such column on `events`, and rejects the query.
(e) The transaction manager, through its lock manager: a lost update is a concurrency anomaly, not a storage or query-processing fault.

Solution: C6. Discussion

(a) Three reasons, each with its failure under the OS alone.

1. **Access-pattern knowledge.** The DBMS knows a sequential scan is one-pass and can protect hot pages (index roots, small dimension tables) from being evicted by it. The OS sees anonymous reads and would flush the useful pages (sequential flooding), so the next lookups miss.
2. **Write ordering for durability.** Recovery requires the log record to reach disk before the data page it describes (write-ahead logging). The OS flushes cached pages in whatever order it likes; a crash between the two writes leaves a data page on disk with no log record to undo it, and recovery is impossible.
3. **Control over eviction and pinning.** An operator working on a page must know it will stay in memory (pinned) until it is done, and the DBMS wants to choose the victim (LRU variants, clock) with knowledge of the query plan. The OS can page out anything at any time, and keeping a copy in both caches wastes memory.

(b) $\log_2 N$ is the depth of a *binary* tree. A B+ tree node is a whole page holding hundreds of keys, so the height is $\log_f N$ with fanout f in the hundreds: for $f = 100$, $\lceil \log_{100} 10^6 \rceil = 3$. Add one heap read: about 4 page reads, not 20. Each page read eliminates a factor of not 2.

Summary: If you missed these, reread

- A3, A5–A9, B5, C5: Chapter 1, Section 3 (the life of a query) and the `EXPLAIN` technique box.
- A11–A20, B1, B2, B8, C1, C2, C6(a): Chapter 2, Sections 2–4 and 6.2.
- A21–A30, B3, B4, B6, B7, C3, C4, C6(b): Chapter 3, Sections 2–5.